BACCALAURÉAT GÉNÉRAL

ÉPREUVE D'ENSEIGNEMENT DE SPÉCIALITÉ

SESSION 2026

NUMÉRIQUE ET SCIENCES INFORMATIQUES

ÉPREUVE DU MARDI 08 SEPTEMBRE 2026

Durée de l'épreuve : **3 heures 30**

L'usage de la calculatrice n'est pas autorisé.

Dès que ce sujet vous est remis, assurez-vous qu'il est complet.

Ce sujet comporte 16 pages numérotées de 1/16 à 16/16.

Le sujet est composé de trois exercices indépendants.

Le candidat traite les trois exercices.

Exercice 1 (6 points)

Cet exercice porte sur les chaînes de caractères et les piles.

Tous les langages de programmation disposent d'expressions. Par exemple :

- l'expression `3 - 11 * 2` s'évalue à -19 ;
- et l'expression `(3 - 11) * 2` s'évalue à -16.

Cette notation, bien que très lisible pour un humain, n'est pas pratique pour l'évaluation automatisée, notamment à cause du parenthésage et de la priorité des opérateurs. On peut alors utiliser la *notation polonaise inverse*. L'idée de cette notation est d'écrire chaque opérateur *après* les termes de cet opérateur, par exemple on écrit `8 2 /` à la place de `8 / 2`. Ainsi en notation polonaise inverse :

- `3 - 11 * 2` s'écrit `3 11 2 * -`.
- `(3 - 11) * 2` s'écrit `3 11 - 2 *`.

On se restreint aux quatre opérateurs de base `+`, `-`, `*` et `/` où l'opérateur `/` donne le quotient de la division euclidienne. On ne manipule que des nombres entiers.

Partie A - Gestion des termes et opérateurs de l'expression

Il faut pouvoir tester si une chaîne de caractères représente un opérateur. On souhaite donc écrire la fonction `est_operateur` suivante où `s` est une chaîne de caractères et qui renvoie un booléen :

```
1 def est_operateur(s):
2     """Renvoie si s correspond à une des 4 opérations
3     ou non."""
4     return len(s) == 1 and (s[0] == '+' ...)
```

1. Recopier et compléter la ligne 4 de la fonction `est_operateur`.

On souhaite maintenant écrire une fonction `est_nombre` qui prend en paramètre une chaîne de caractères `s` et qui renvoie si `s` correspond à un nombre (positif ou négatif). Par exemple les appels à `est_nombre('-421')` et `est_nombre('421')` renvoient `True` tandis que `est_nombre('*')` renvoie `False`.

2. Donner une chaîne de caractères `s_faux` telle que `est_operateur(s_faux)` et `est_nombre(s_faux)` renvoient `False` tous les deux.

On suppose qu'on dispose d'une fonction `est_chiffre` qui prend en paramètre une chaîne de caractères `s` *de longueur 1*, et qui renvoie si `s` correspond à un chiffre (0, 1, 2, etc, jusqu'à 9) ou non. Par exemple l'appel à `est_chiffre('5')` renvoie `True` et celui à `est_chiffre('+')` renvoie `False`.

Pour simplifier l'écriture de la fonction `est_nombre`, on souhaite d'abord écrire une fonction `est_nombre_positif` qui prend en paramètre une chaîne de caractères non vide `s` et un indice de début `deb` et qui renvoie si la chaîne `s` à partir de l'indice `deb` inclus ne contient que des chiffres. On suppose que le paramètre `deb` est positif et strictement inférieur à la longueur de la chaîne `s`. Voici des exemples d'appels à la fonction `est_nombre_positif` :

- l'appel à `est_nombre_positif('421', 0)` renvoie `True` ;
- l'appel à `est_nombre_positif('+', 0)` renvoie `False` ;
- l'appel à `est_nombre_positif('+421', 0)` renvoie `False` car le caractère `'+'` n'est pas un chiffre ;
- l'appel à `est_nombre_positif('-421', 0)` renvoie `False` car le caractère `'-'` n'est pas un chiffre ;
- l'appel à `est_nombre_positif('-421', 1)` renvoie `True` car on démarre la vérification à partir du caractère `'4'`.

3. Écrire la fonction `est_nombre_positif` en utilisant `est_chiffre`.

On donne maintenant l'écriture de la fonction `est_nombre`

```
1 def est_nombre(s):
2     debut = 0
3     if s[0] == '-':
4         debut = 1
5     if debut < len(s):
6         return est_nombre_positif(s, debut)
7     else:
8         return False
```

On suppose que les expressions sont des chaînes de caractères dont les termes et les opérateurs sont deux à deux séparés par un espace comme par exemple `'3 11 - 2 *'`. On suppose qu'il n'y a pas d'espace à la fin de la chaîne. On souhaite obtenir une liste des termes et opérateurs de l'expression à l'aide d'une fonction `separe_elements`. Par exemple, l'appel à `separe_elements('5 67 +')` renvoie la liste `['5', '67', '+']`.

On donne une écriture partielle de cette fonction.

```
1 def separe_elements(expression):
2     sortie = []
3     chaine = ''
4     for c in expression:
5         if ...:
6             chaine = chaine + c
7         else:
8             sortie.append(chaine)
```

```
 9             chaine = ...
10     ...
11     return sortie
```

4. Recopier et compléter sur la copie les lignes 5, 9 et 10 de la fonction `separe_elements`.

Pour préparer la partie B, il faut placer les termes et opérateurs d'une liste sur une pile. On dispose d'une classe `Pile` complète dont les méthodes sont décrites ci-dessous.

```
 1 class Pile:
 2     def __init__(self):
 3         """Initialise le contenu d'une pile vide."""
 4
 5     def est_vide(self):
 6         """Renvoie un booléen indiquant si la pile est
 7            vide ou non."""
 8
 9     def empile(self, elt):
10         """Ajoute l'élément elt sur le haut de la pile."""
11
12     def depile(self):
13         """Supprime le haut de la pile et le renvoie.
14         La pile doit être non vide avant l'appel."""
15
16     def examine(self):
17         """Renvoie le haut de la pile sans le supprimer."""
```

L'instruction `p = Pile()` permet d'initialiser une pile vide dans la variable `p`.

On souhaite écrire une fonction `conversion_pile` qui prend en paramètre une liste de chaînes de caractères et renvoie une pile contenant ces chaînes. La première chaîne de la liste doit être placée au sommet de la pile (et la dernière au fond).

Lors de la conversion, la fonction vérifie que chacune des chaînes de la liste est bien soit un nombre, soit un opérateur. Si ce n'est pas le cas, alors la fonction renvoie `None`. À ce stade, on ne s'occupe pas de vérifier si l'expression est syntaxiquement correcte. Voici des exemples d'appels à la fonction `conversion_pile` :

- l'appel à `conversion_pile(['5', '67', '+'])` renvoie une pile avec `'+'` au fond, suivi de `'67'`, suivi de `'5'` au sommet ;
- l'appel à `conversion_pile(['5', '6'])` renvoie une pile avec `'6'` au fond, suivi de `'5'` au sommet ;
- l'appel à `conversion_pile(['5', 'abc'])` renvoie `None` car un des éléments de la liste n'est ni un nombre ni un opérateur.

5. Écrire la fonction `conversion_pile`.

Partie B - Évaluation d'une expression en notation polonaise inverse

Dans cette partie, on suppose que les expressions sont données en notation polonaise inverse et sont syntaxiquement correctes. L'évaluation d'une expression se fait alors à l'aide de deux piles.

- La *pile d'entrée* est obtenue grâce à la fonction `conversion_pile` de la partie A. Elle peut contenir à la fois des nombres et des opérateurs. On rappelle que le sommet de la pile contient initialement le début de l'expression.
- La *pile de calcul* contient les résultats des calculs intermédiaires. Elle ne contient donc que des nombres sous forme de chaînes de caractères. Initialement cette pile est vide.

Pour évaluer une expression à l'aide de ces deux piles, on suit l'algorithme suivant.

Tant que la pile d'entrée n'est pas vide, on regarde son sommet :

- si c'est un nombre alors on le dépile puis on l'empile sur la pile de calcul ;
- si c'est un opérateur alors on le dépile puis on dépile les deux éléments au sommet de la pile de calcul, on effectue l'opération correspondante, puis on place le résultat sur le haut de la pile de calcul.

Quand la pile d'entrée est vide, l'évaluation est terminée. On lit alors le résultat sur le sommet de la pile de calcul.

La figure 1 ci-après montre l'état de la pile d'entrée et de la pile de calcul à chaque étape de cet algorithme lors de l'évaluation de de l'expression `3 11 - 2 *`.

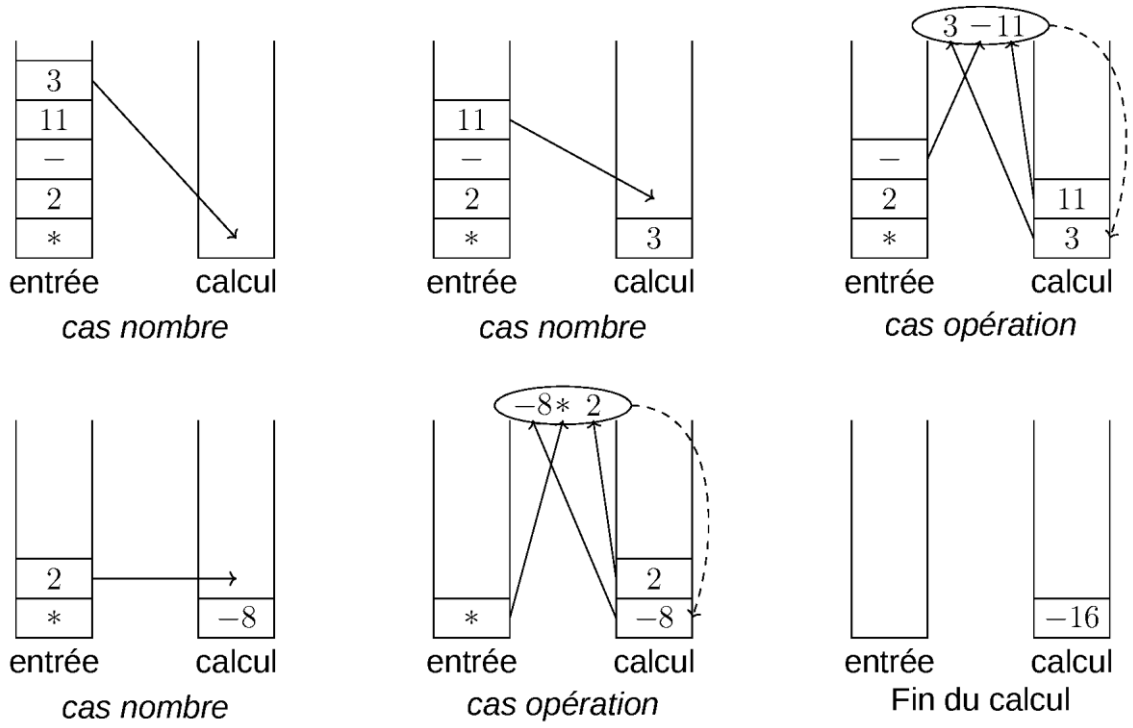


Figure 1. Déroulement de l'algorithme d'évaluation de l'expression `3 11 - 2 *`

6. Dessiner les états successifs de la pile de calcul lors de l'évaluation de l'expression `3 11 2 * -`. Le dessin de la pile d'entrée n'est pas demandé.

On suppose que la pile d'entrée et la pile de calcul sont respectivement stockées dans des variables `p_entree` et `p_calcul`.

7. Donner l'instruction Python permettant, à la fin de l'évaluation, d'obtenir le résultat.

On donne ci-dessous la fonction `traite_nombre` qui implémente une étape de l'algorithme d'évaluation dans le cas où le sommet de la pile d'entrée est un nombre.

```
1 def traite_nombre(p_entree, p_calcul):
2     nombre = p_entree.depile()
3     p_calcul.empile(nombre)
```

On donne une fonction `calcule(a, op, b)` qui prend en paramètres des *chaînes de caractères*, représentant respectivement un nombre, un opérateur et un nombre, et qui renvoie le résultat de l'opération sous forme de chaine de caractères. Par exemple l'appel à `calcule('2', '+', '6')` renvoie `'8'`, tandis que l'appel à `calcule('3', '-', '5')` renvoie `'-2'`.

On souhaite écrire une fonction `traite_operateur` qui implémente une étape de l'évaluation dans le cas où le sommet de la pile d'entrée est un opérateur.

```
1 def traite_operateur(p_entree, p_calcul):
2     operateur = ...
3     ... = p_calcul.depile()
```

```
4       ... = p_calcul.depile()
5       res = calcule(terme1, operateur, terme2)
6       ...
```

8. Recopier puis compléter la fonction `traite_operateur`.

9. Écrire une fonction `evaluation(expression)` qui prend en paramètre une expression en notation polonaise inverse dont les termes et les opérateurs sont deux à deux séparés par des espaces et qui renvoie l'évaluation de cette expression sous forme de chaîne de caractères.

Exercice 2 (6 points)

Cet exercice porte sur la récursivité, les arbres binaires et la programmation orientée objet.

Le format ZIP est un format de fichier compressé que l'on utilise pour archiver ou transmettre des informations. Dans sa première version, ce format utilisait le codage de Shannon-Fano pour la compression. On se propose d'étudier ce codage dans cet exercice.

On donne une partie d'une table ASCII (American Standard Code for Information Interchange) où les caractères sont codés sur 8 bits.

Caractère	Code ASCII (Décimal)	Code ASCII (Hexadécimal)
`A`	65	41
`B`	66	42
`C`	67	43
`D`	68	44

1. Donner le code binaire du caractère `B`.

Avec le codage précédent, un texte de sept caractères comme `Bonjour` nécessite $7 \times 8 = 56$ bits, plus généralement un texte de n caractères nécessite $8 \times n$ bits.

L'algorithme suivant, basé sur le codage de Shannon-Fano, permet de réduire ce nombre de bits. On l'illustre pas à pas sur le texte `Bonjour`.

Étape 1 : Calcul du nombre d'occurrences dans le texte de chaque caractère.

Caractère	`B`	`o`	`n`	`j`	`u`	`r`	Total (poids)
Nombre d'occurrences	1	2	1	1	1	1	7

On définit le *poids* d'un tableau de caractères comme étant la somme des nombres d'occurrences de ses caractères.

Étape 2 : Réalisation d'un tri par ordre décroissant du nombre d'occurrences.

Lorsque plusieurs caractères ont le même nombre d'occurrences, on les classe selon leur ordre d'apparition dans le texte. Par exemple `B` apparaît avant `n` dans le texte `Bonjour`, il est donc placé avant dans le tableau ci-dessous.

Caractère	o	B	n	j	u	r	Total (poids)
Nombre d'occurrences	2	1	1	1	1	1	7

Étape 3 : Construction d'un arbre de codage.

Un *arbre de codage* est un arbre binaire dont les feuilles sont des caractères deux à deux distincts.

L'arbre de codage est construit grâce au tableau trié du nombre d'occurrences. Ce tableau de caractères est placé en racine de l'arbre. À chaque étape de la construction, le tableau de caractères du nœud courant `tab` est coupé en deux sous-tableaux de poids équilibrés, sauf si `tab` contient un seul caractère, auquel cas c'est une feuille. Plus précisément, pour découper `tab` l'algorithme étudié commence par placer le caractère de gauche dans un sous-tableau `tab_gauche`, puis fait de même pour le caractère suivant tant que le poids de `tab_gauche` ne dépasse pas le poids des caractères restants. Les caractères restants sont alors placés dans un sous-tableau `tab_droit`.

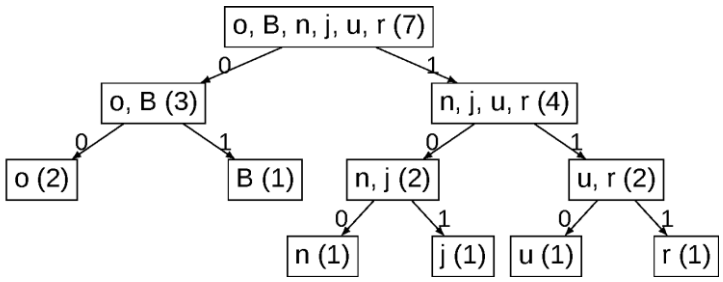


Figure 1. Arbre de codage obtenu pour le texte `Bonjour`.

Le *code* d'un caractère qui apparaît dans une feuille est donné par le chemin qui mène de la racine à cette feuille, où `0` indique "à gauche" et `1` indique "à droite". Par exemple, dans l'arbre de codage de la figure 1, le code de `n` est `100`.

Étape 4 : Codage du texte en binaire selon l'arbre de codage.

Le codage des trois premiers caractères `Bon` de `Bonjour` est ainsi `01 00 100`, c'est-à-dire `0100100` ce qui nécessite 7 bits au lieu de $3 \times 8 = 24$ bits.

2. Coder en binaire le mot `Bonjour` en utilisant la méthode de l'étape 4 avec l'arbre de codage de la figure 1.

3. Écrire le tableau par ordre décroissant des nombres d'occurrences de chaque caractère du texte `Perrine`, puis dessiner l'arbre de codage correspondant.

On donne ci-dessous une partie du code de la fonction `calculer_nb_occ` qui prend en paramètre une chaîne de caractères `texte` et qui renvoie un dictionnaire

dont les clés sont les caractères de `texte`, et les valeurs associées leurs nombres d'occurrences dans `texte`.

```
1 def calculer_nb_occ(texte):
2     dico_occ = ...
3     for c in texte:
4         if c in dico_occ.keys():
5             ...
6         else:
7             ...
8     return dico_occ
```

Par exemple, l'appel `calculer_nb_occ('Shannon-Fano')` renvoie le dictionnaire suivant.

```
{'S': 1, 'h': 1, 'a': 2, 'n': 4, 'o': 2, '-': 1, 'F': 1}
```

4. Recopier et compléter les lignes 2, 5 et 7 de la fonction `calculer_nb_occ`.

On suppose qu'on dispose d'une fonction `creer_tab_car` qui prend en paramètre un dictionnaire associant à des caractères leur nombre d'occurrences (comme celui renvoyé par un appel à `calculer_nb_occ`), et qui le transforme en un tableau de couples (caractère, nombre d'occurrences). Dans la suite, on appellera *tableau de caractères* de tels tableaux de couples.

On suppose qu'on dispose aussi d'une fonction `tri_fusion` qui trie un tableau de caractères par nombre d'occurrences décroissant (autrement dit les tuples sont triés dans l'ordre décroissant de leur deuxième composante).

On dispose enfin de la fonction `calculer_poids` qui calcule le poids d'un tableau de caractères.

Des exemples d'appels à ces fonctions sont reproduits ci-après.

```
>>> texte_ex = 'Shannon-Fano'
>>> dico_occ_ex = calculer_nb_occ(texte_ex)
>>> dico_occ_ex
{'S': 1, 'h': 1, 'a': 2, 'n': 4, 'o': 2, '-': 1, 'F': 1}

>>> tab_non_trie_ex = creer_tab_car(dico_occ_ex)
>>> tab_non_trie_ex
[('S', 1), ('h', 1), ('a', 2), ('n', 4), ('o', 2), ('-', 1),
('F', 1)]

>>> tab_trie_ex = tri_fusion(tab_non_trie_ex)
>>> tab_trie_ex
[('n', 4), ('a', 2), ('o', 2), ('S', 1), ('h', 1), ('-', 1),
('F', 1)]
```

```
>>> calculer_poids(tab_trie_ex)
12
```

On donne ci-dessous le code de la fonction `couper` qui prend en argument un tableau de caractères et qui renvoie l'indice qui permet de le couper en deux sous-tableaux équilibrés selon le principe de l'étape 3.

```
1 def couper(tab):
2     poids = calculer_poids(tab)
3     demi_poids = poids // 2
4     s = 0
5     i = 0
6     while s < demi_poids:
7         s = s + tab[i][1]
8         i = i + 1
9     return i
```

5. Donner et justifier la valeur renvoyée par la fonction couper pour le tableau de caractères suivant.

```
[('n', 4), ('a', 2), ('o', 2), ('S', 1), ('h', 1), ('-', 1), ('F', 1)]
```

Afin d'implémenter les arbres de codage, on se munit de la classe `Noeud` ci-dessous, où l'attribut `racine` est un tableau de caractères, et où les attributs `gauche` et `droit` donnent les éventuels sous-arbres du nœud.

```
1 class Noeud:
2     def __init__(self, gauche=None, valeur=None, droit=None):
3         self.gauche = gauche
4         self.valeur = valeur
5         self.droit = droit
6 
7     def est_feuille(self):
8         return self.gauche is None and self.droit is None
```

On donne une partie du code de la fonction `creer_arbre` qui réalise l'étape 3.

```
1 def creer_arbre(tab_trie):
2     # Cas de base : une feuille
3     if len(tab_trie) == 1:
4         caractere = ...
5         return Noeud(None, caractere, None)
6 
7     # Calculer la coupure
8     i = couper(tab_trie)
9 
10    # Répartir les caractères
11    tab_gauche = ...
12    tab_droit = ...
13
```

```
14      gauche = creer_arbre(tab_gauche)
15      droit = creer_arbre(tab_droit)
16
17      # Renvoyer le nœud interne
18      return Noeud(gauche, None, droit)
```

6. Justifier que la fonction creer_arbre est une fonction récursive.

7. Recopier et compléter la ligne 4 de la fonction creer_arbre pour récupérer le caractère à placer en feuille.

8. Recopier et compléter les lignes 11 et 12 de la fonction `creer_arbre` pour répartir les caractères dans deux tableaux en fonction de `i`.

On donne une partie du code de la fonction `decoder_avec_arbre` qui permet de décoder un code binaire à partir de l'arbre de codage, et renvoyer le texte initial. Cette fonction parcourt l'arbre binaire de codage en partant de la racine, en suivant chaque caractère de `texte_code`. Quand le parcours atteint une feuille de l'arbre, on note le caractère ainsi atteint et on poursuit le décodage des caractères suivants en repartant de la racine de l'arbre.

```
 1 def decoder_avec_arbre(texte_code, racine):
 2     resultat = ''
 3     noeud_courant = ...
 4     for bit in texte_code:
 5         if bit == '0':
 6             noeud_courant = ...
 7         else:
 8             noeud_courant = ...
 9
10         if noeud_courant.est_feuille():
11             resultat = resultat + ...
12             noeud_courant = ...
13     return resultat
```

On poursuit ci-dessous les exemples donnés entre la question 4 et la question 5.

```
>>> arbre_ex = creer_arbre(tab_trie_ex)
>>> codage_ex = coder_avec_arbre(texte_ex, arbre)
>>> codage_ex
'10111001000010000111011110100100'
>>> decoder_avec_arbre(codage_ex, arbre)
'Shannon-Fano'
```

9. Recopier et compléter les lignes 3, 6, 8, 11 et 12 de la fonction `decoder_avec_arbre`.

Exercice 3 (8 points)

Cet exercice porte sur la programmation orientée objet en Python, les bases de données et le langage SQL. Il est constitué de deux parties indépendantes.

Partie A

Dans cette partie il est question de la classe `CompteBancaire` permettant d'implémenter les diverses opérations réalisables sur un compte bancaire. On rappelle que *débiter* un compte signifie retirer de l'argent du compte, tandis que *créditer* un compte signifie ajouter de l'argent sur le compte.

Les objets de la classe `CompteBancaire` possèdent trois attributs :

- `nom` une chaîne de caractères correspondant au nom du propriétaire du compte ;
- `numero` un entier identifiant le compte bancaire ;
- `solde` un flottant représentant le montant disponible en euros sur le compte.

Les objets de la classe `CompteBancaire` possèdent aussi plusieurs méthodes :

- `__init__` qui en plus de `self` a pour paramètres `nom`, `numero` et `solde_initial` correspondant aux valeurs initiales à affecter respectivement aux attributs `nom`, `numero` et `solde` ;
- les trois méthodes `get_nom`, `get_numero` et `get_solde` qui renvoient respectivement les valeurs des attributs `nom`, `numero` et `solde`.

1. Recopier et compléter la méthode `__init__`.

```
1 class CompteBancaire:
2     def __init__(...):
3         ...
4         ...
5         ...
```

2. Donner l'instruction permettant de créer `compte_cara` un compte bancaire de numéro `15083` et de montant initial de 670 € au nom de `Cara`.

3. Écrire la méthode `crediter` de la classe `CompteBancaire` qui prend en paramètre `montant` un montant en euros et qui ajoute ce montant au solde du compte.

Dans la suite on suppose que la classe `CompteBancaire` dispose aussi d'une méthode `debiter` qui prend en paramètre `montant` un montant en euros et qui retire ce montant au solde du compte. De plus, on veillera à n'utiliser que la méthode `get_solde` pour obtenir le solde d'un compte, et à n'utiliser que les méthodes `crediter` et `debiter` pour le modifier.

4. Écrire l'instruction à exécuter pour créditer de 100 € le compte de Cara créé à la question 2.

On souhaite maintenant effectuer des virements d'un compte vers un autre compte.

5. Écrire la méthode `effectuer_virement` qui prend en paramètres :

   – `compte_destinataire` un objet du type `CompteBancaire` ;

   – `somme_a_virer` un nombre flottant représentant la somme à virer en euros ; et qui, lorsque le montant disponible le permet, retire la somme indiquée du compte courant, et crédite le compte destinataire de cette même somme. Cette fonction doit en outre renvoyer `True` lorsque le virement a pu être effectué et `False` sinon.

On considère dans la suite des fonctions opérant sur des listes de comptes bancaires, c'est-à-dire des objets de type `list` dont les éléments sont des instances de la classe `CompteBancaire`.

6. Écrire une fonction `appliquer_frais_de_gestion` qui prend en paramètres une liste de comptes comptes et un flottant `cout` représentant le montant des frais de gestion en euros, et qui débite ce montant de chaque compte de la liste.

7. Écrire une fonction `comptes_dans_le_rouge` qui prend en paramètre comptes une liste de comptes et qui renvoie la liste des comptes de cette liste dont le montant est strictement négatif.

8. Recopier et compléter la fonction `trier_comptes` qui prend en paramètre une liste de comptes comptes et qui trie ces comptes dans l'ordre **décroissant** de leurs soldes. Ce tri se fait en place, c'est-à-dire en modifiant directement le contenu de la liste `comptes`.

```
1 def trier_comptes(comptes):
2     for i in range(len(comptes) - 1):
3         jmax = i
4         for j in range(i + 1, len(comptes)):
5             if ...:
6                 jmax = j
7         # échange des cases
8         ...
```

9. Donner le nom de l'algorithme de tri utilisé à la question précédente.

10. Donner le nom d'un algorithme de tri plus efficace étudié en terminale, et nommer le paradigme algorithmique mis en œuvre dans cet algorithme plus efficace.

Partie B

Dans cette partie, on pourra utiliser les clauses du langage SQL pour :

- construire des requêtes d'interrogation à l'aide de `SELECT`, `FROM`, `WHERE` et `JOIN ... ON` ;
- construire des requêtes d'insertion et de mise à jour à l'aide de `UPDATE`, `INSERT` et `DELETE` ;
- réaliser des agrégations à l'aide de `COUNT`, `MIN`, `MAX`, `SUM` et `AVG` ; telle que `SELECT COUNT(numero) FROM compte;` pour obtenir le nombre de comptes enregistrés.

Les informations concernant les clients de la banque en ligne NéoBanque, leurs comptes ainsi que les virements d'argent effectués entre ces comptes ont été enregistrés dans une base de données, dont le schéma relationnel est précisé ci-dessous.

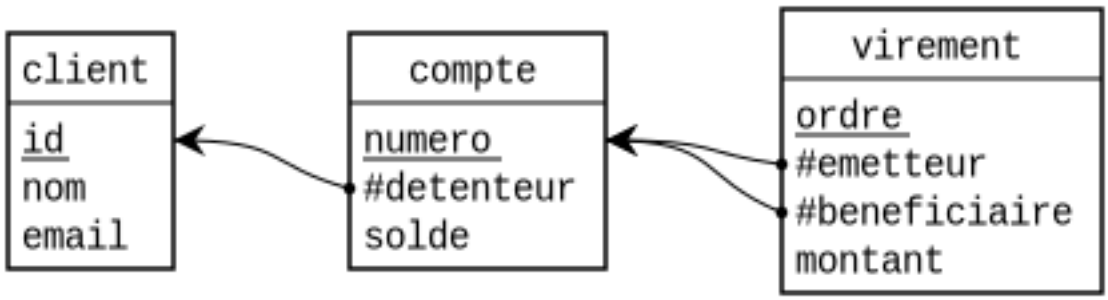


Figure 1. Schéma relationnel de la base de données

On rappelle que dans ce schéma, une clé primaire est soulignée et une clé étrangère est précédée du caractère #. Il s'ensuit que :

- la clé étrangère `detenteur` de la relation `compte` fait référence à la clé primaire `id` de la relation `client` ;
- les clés étrangères `emetteur` et `beneficiaire` de la relation `virement` font référence à la clé primaire `numero` de la relation `compte`.

On précise par ailleurs la signification de quelques attributs :

- `solde` correspond à l'argent disponible sur un compte ;
- `emetteur` est le compte débité du `montant` indiqué à l'occasion d'un virement ;
- `beneficiaire` est le compte crédité du `montant` indiqué à l'occasion de ce même virement.

Le client dont l'identifiant est `3512` a changé d'adresse mail ; sa nouvelle adresse est désormais "`alain.magot@client.gagnant`".

11. Donner la requête SQL effectuant cette modification d'adresse dans la base de données.

12. Rédiger la requête SQL calculant le montant total des virements émis par le détenteur du compte numéro `123456`.

13. Écrire la requête SQL permettant d'obtenir la table des numéros des comptes appartenant aux personnes portant le nom `Richard Oise` au moyen d'une jointure.

On souhaite enregistrer un virement de 281,65 € du compte `304050` vers le compte `908070` ayant pour numéro d'ordre `1122`.

14. Rédiger la requête SQL correspondante. (Il ne s'agit pas de débiter ni de créditer les comptes en question, seulement d'enregistrer le virement.)

Le client d'identifiant `3512`, peu satisfait des prestations, a décidé de quitter la banque.

15. Donner les raisons expliquant l'échec de la requête suivante et expliquer comment on pourrait y remédier.

```
DELETE FROM client WHERE id = 3512;
```